

Project Document: Full-Stack LLM Chatbot Application

1. Project Title

Building a Full-Stack Chatbot using FastAPI and LLM APIs

2. Project Description

In this project, students will develop a full-stack chatbot web application that communicates with a Large Language Model (LLM) via API. The application will allow users to ask questions, receive AI-generated responses, and view past conversation history.

The project is divided into two main parts:

- Frontend (HTML, CSS, minimal JavaScript)
- Backend (FastAPI, API integration, database)

3. Objectives

By completing this project, students will:

- Understand how frontend and backend communicate
- Learn to build APIs using FastAPI
- Integrate external LLM APIs
- Work with databases (MongoDB or SQL)
- Manage environment variables securely
- Design and structure a real-world project

4. Core Functional Requirements

4.1 Chat Functionality

- User enters a question
- Backend sends it to LLM API
- Response is returned and displayed

4.2 Chat Storage

- Each chat must store:
 - Question
 - Answer
 - Timestamp

4.3 History Feature

- Display list of previously asked questions
- Each item should be clickable

4.4 Chat Detail View

- Clicking a history item should open full chat
- Show both question and answer

5. System Architecture

Frontend:

- HTML → Structure
- CSS → Styling
- JavaScript → API calls (Fetch API)

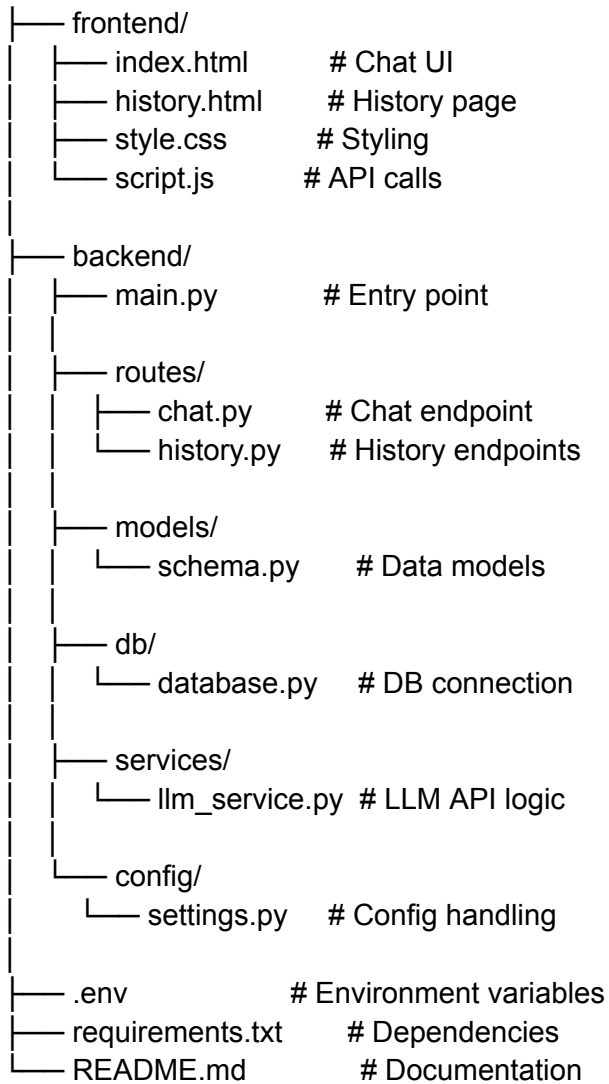
Backend:

- FastAPI server
- API routes
- LLM API integration

- Database connection

6. Project Structure

project-root/



7. Environment Configuration

Students must use a .env file to store sensitive data:

Example:

```
API_KEY=your_llm_api_key DB_URL=your_database_url
```

Rules:

- Do NOT hardcode API keys
- Use environment variable loaders

8. API Design

8.1 Chat Endpoint

POST /chat

Request:

```
json { "question": "What is AI?" }
```

Response:

```
json { "answer": "AI stands for Artificial Intelligence..." }
```

8.2 Get History

GET /history

Response:

```
json [ { "id": "1", "question": "What is AI?" } ]
```

8.3 Get Chat by ID

GET /chat/{id}

Response:

```
json { "question": "What is AI?", "answer": "AI is..." }
```

9. Database Design

Option 1: MongoDB

Collection: chats

```
json { "_id": "unique_id", "question": "string", "answer": "string", "timestamp": "datetime" }
```

Option 2: SQL

```
| id | question | answer | timestamp |  
|----|-----|-----|-----|
```

10. Application Flow

Backend Flow:

1. Receive request
2. Extract user question
3. Send request to LLM API
4. Receive response
5. Store in database
6. Return response

Frontend Flow:

1. User enters input
2. JavaScript sends API request
3. Backend processes request
4. Response displayed to user

11. 🚀 Development Timeline (5–7 Days)

Day 1:

- Project setup
- FastAPI installation
- Basic server

Day 2:

- Chat API creation
- LLM API integration

Day 3:

- Database setup
- Save chat data

Day 4:

- History API

Day 5:

- Frontend UI

Day 6:

- Frontend-backend integration

Day 7:

- Testing and improvements

12. 🚀 Bonus Features (Optional)

- Streaming responses
- Delete chat history
- Search functionality
- UI improvements (dark mode)